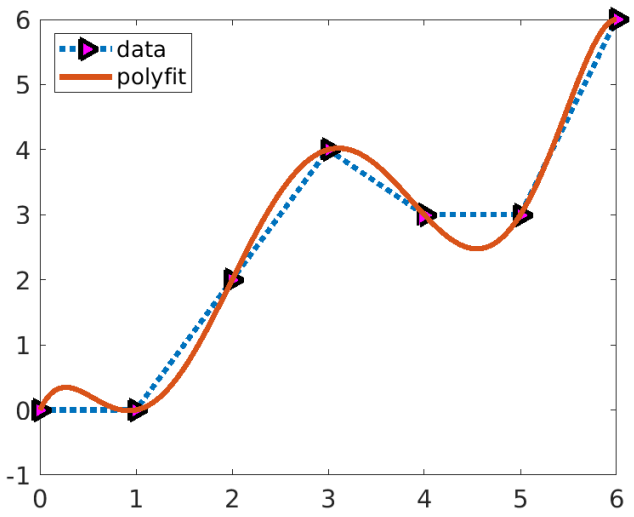
CS319: Scientific Computing (with MATLAB)

DRAFT : Fitting and Files.

Niall Madden

Week 6: **9am and 4pm**, 15 Feb 2023



Important: you should read:

- Section 7.5 of Learning MATLAB: https://doi-org.nuigalway.idm.oclc.org/10.1137/1.9780898717662
- Chapter 11.1 of The MATLAB Guide: https://doi-org.nuigalway.idm.oclc.org/10.1137/1.9781611974669

This week, in CS319:

1. 1: Data Fitting with Polynomials
   - Polynomial interpolation
   - polyfit
   - Data
   - Functions
2. 2: Least Squares
   - How does it work?
3. In-class test
   - Sample questions

For more, see

- Chapter 5 of Learning MATLAB: https://doi-org.nuigalway.idm.oclc.org/10.1137/1.9780898717662
- Chapter 8 of The MATLAB Guide: https://doi-org.nuigalway.idm.oclc.org/10.1137/1.9781611974669

Preview

...

We finished Week 5 by introducing **polynomial interpolation**. The two problems of interest are:

Data interpolation: given a set of $n+1$ points in 2D, find a polynomial of degree n that goes through them.

Function interpolation: given function find a polynomial that agrees with it at n points.

Questions: Why do this?

- To evaluate the polynomial at multiple other points;
- Estimate derivatives;
- Estimate integral;
- ...

Question: How can we do this? **Answer:** `polyfit()`

In the following examples, we will construct the polynomial interpolant using the function `polyfit()`.

Syntax: `p = polyfit(x, y, n);`
where `x` and `y` are vectors of the same length, and n is a integer.

If `n = length(x)-1`, then it should interpolate the data. For smaller n, a least-squares fit is used (more of that later).

`p` is a vector with $n+1$ entries, where *`p(i)`* is the coefficient of x^{n+1-i} in the polynomial. That is, it represents the polynomial

$$p(1)x^n + p(2)x^{n-1} + \cdots + p(n)x + p(n+1).$$

```
>> x=1:3
x =
      1      2      3
>> y = 1-3*x-2*x.^2
y =
      6     17     34
>> p = polyfit(x,y,2)
p =
   -2.0000   -3.0000    1.0000
```

Polynomial Data Interpolation

Given the $n+1$ points (x_0, y_0), (x_1, y_1), $\ldots$, (x_n, y_n), we want to find the polynomial p_n such that

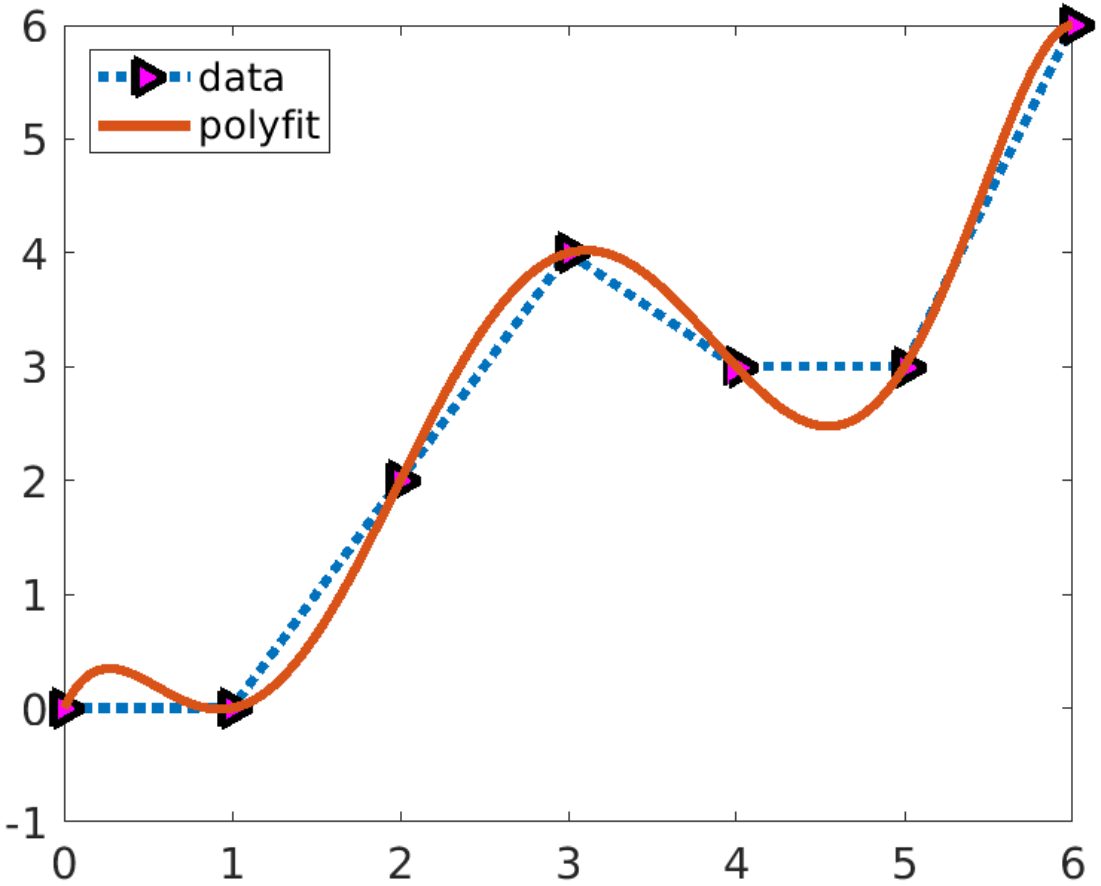
$$p_n(x_0) = y_0, p_n(x_1) = y_1, \ \ldots, \ p_n(x_n) = y_n.$$

PolyDataInterp.m

```
x = 0:6;
y = [0, 0, 2, 4, 3, 3, 6];
p = polyfit(x, y, length(x)-1);

X = linspace(x(1), x(end), 1001); % Points for plotting;
Y = polyval(p, X);
plot(x, y, ':>', X, Y, '-', ...
    'LineWidth',3, 'MarkerSize', 10,...
    'MarkerFaceColor', 'magenta', 'MarkerEdgeColor', 'k');
legend('data', 'polyfit', 'location', 'northwest')
set(gca, 'FontSize', 14)
```

PolyDataInterp.m

```
x = 0:6;
y = [0, 0, 2, 4, 3, 3, 6];
p = polyfit(x, y, length(x)-1);
```



Polynomial Function Interpolation

In practice, this works very similarly to data interpolation: given a set of points x_0, x_1, ..., x_n, and a function f, we compute the interpolant to $(x_0, f(x_0))$, $(x_1, f(x_0))$, ..., $(x_n, f(x_n))$.

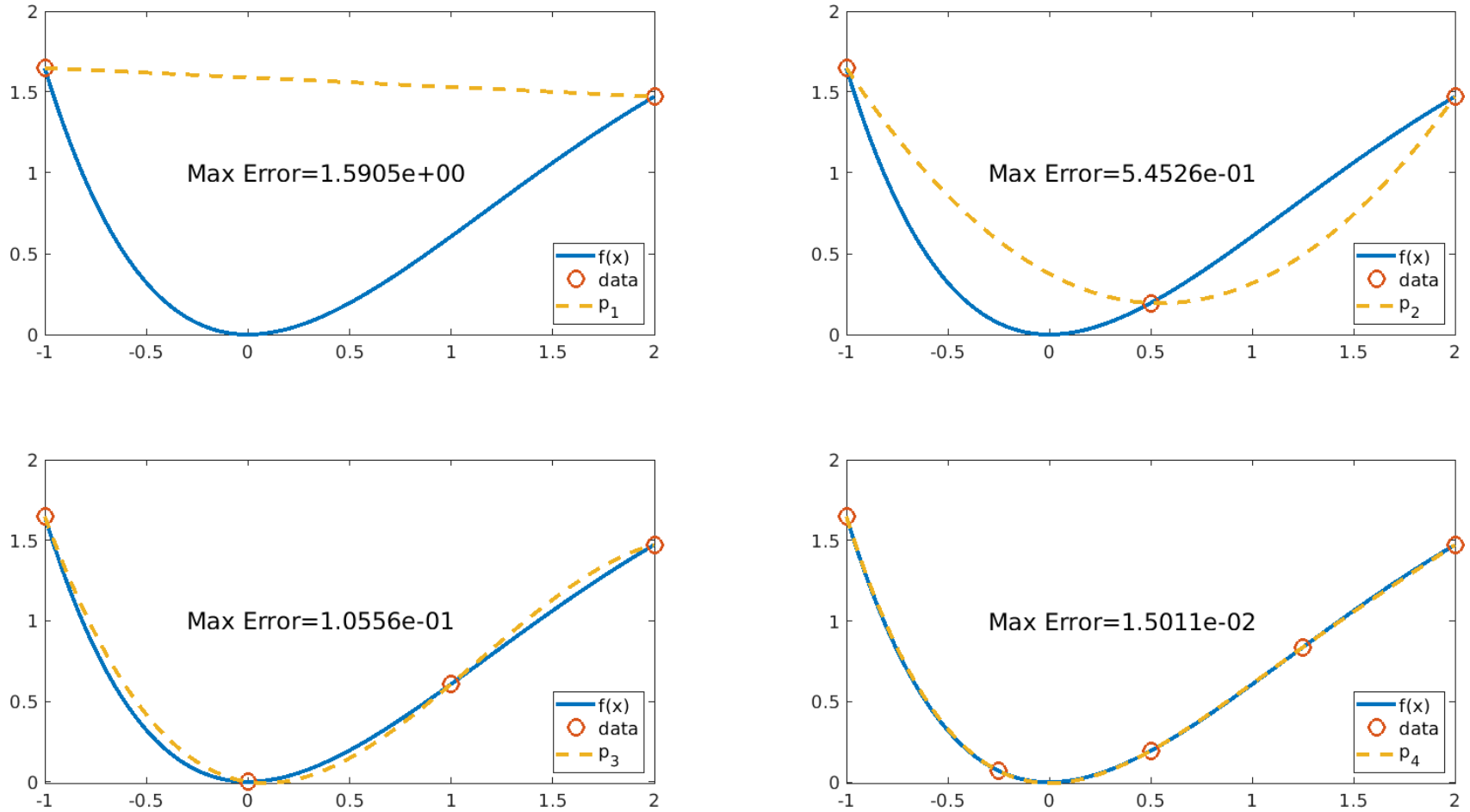
The main differences are

- We are free to choose any points in the function's domain;
- We can vary the number of points easily;
- We can estimate errors: $\|f(x) - p_n(x)\|$.

In this example, we'll construct the polynomial interpolant to $f(x) = x^2 e^{-x/2}$, in the interval $[-1, 2]$.

PolyFunctionInterpolation.m

```
f = @(x)(x.^2).*exp(-x/2);
for n=1:4
    xp = linspace(-1,2,n+1);
    p = polyfit(xp, f(xp), n);

    X = linspace(-1,2, 1001); % Points for plotting;
    Y = polyval(p, X);
    Error = norm(f(X)-Y,'inf');
    fprintf('n=%2d, Error=%10.5e\n', n, Error)
    subplot(2,2,n);
    plot(X, f(X), xp, f(xp), 'o', X, Y, '--', ...
        'LineWidth',3, 'MarkerSize', 12);
    leg_str = sprintf('p_{%d}', n);
    legend('f(x)', 'data', leg_str, 'FontSize', 14, ...
        'location', 'southeast')
    Error_str = sprintf('Max Error=%5.4e', Error);
    text(-0.3,1, Error_str, 'FontSize', 18)
    set(gca, 'FontSize', 14)
end
```


Max Error=1.5905e+00
f(x)
data
p1
Max Error=5.4526e-01
f(x)
data
p2
Max Error=1.0556e-01
f(x)
data
p3
Max Error=1.5011e-02
f(x)
data
p4

2: Least Squares

In all the previous examples,

- We fitted polynomials of degree n to $n+1$ points, and
- There was no noise in the data.

In many applications, we wish to use low-order polynomials,

- because there are more stable;
- the underlying data is noisy (see Lab 3!).

Applications:

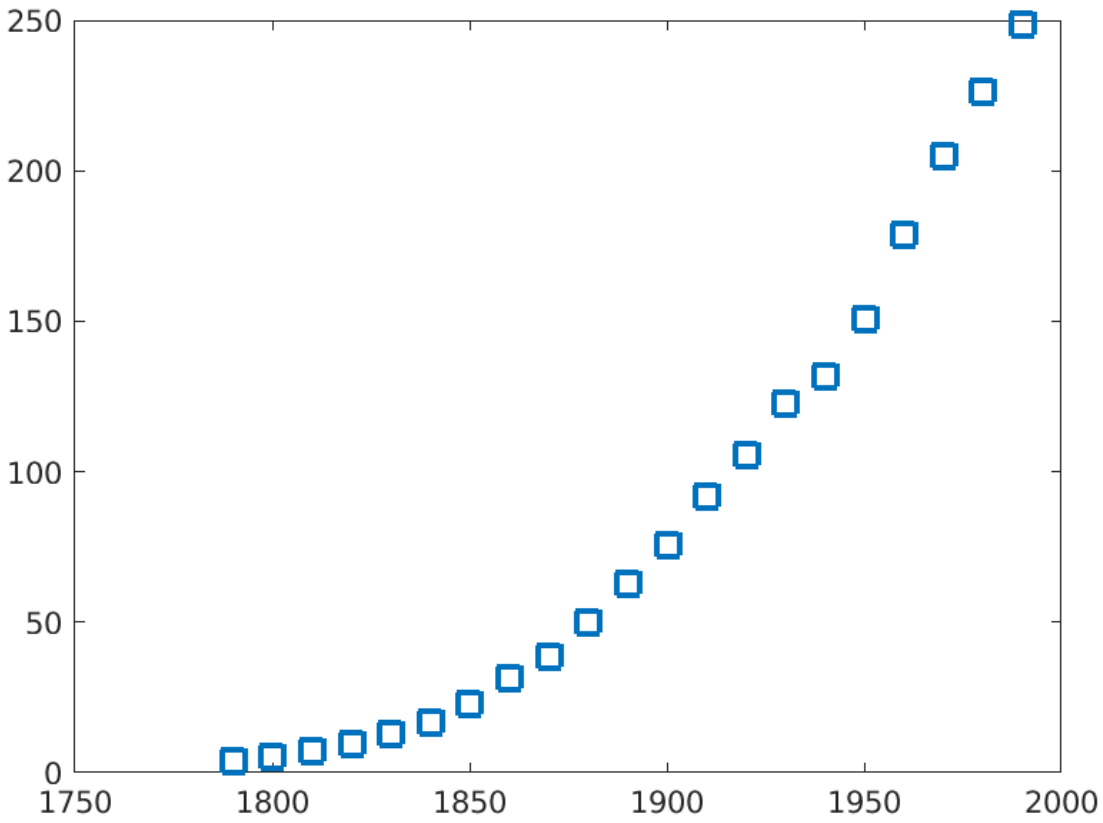
1. Estimating values at non-observed points;
2. Estimating growth rates;
3. **Filtering out noise**.

Example: US Census Data

1. Construct and plot linear, quadratic and cubic fits to the US census data in *census.mat*.
2. For each, estimate the "root mean squared error" ("l^2", or "Euclidian" norm to its friends.
3. For each, what is the estimated rate of growth in 1990?
4. For each, what is the estimated population in 1985 (not a census year)?
5. For each, what is the estimated population in 2000, 2010, and 2020, and how accurate is that?

2: Least Squares

```
load census;
plot(cdate, pop, 's', 'MarkerSize', 10, 'LineWidth', 2);
```



2: Least Squares

We can construct the linear fit as follows:

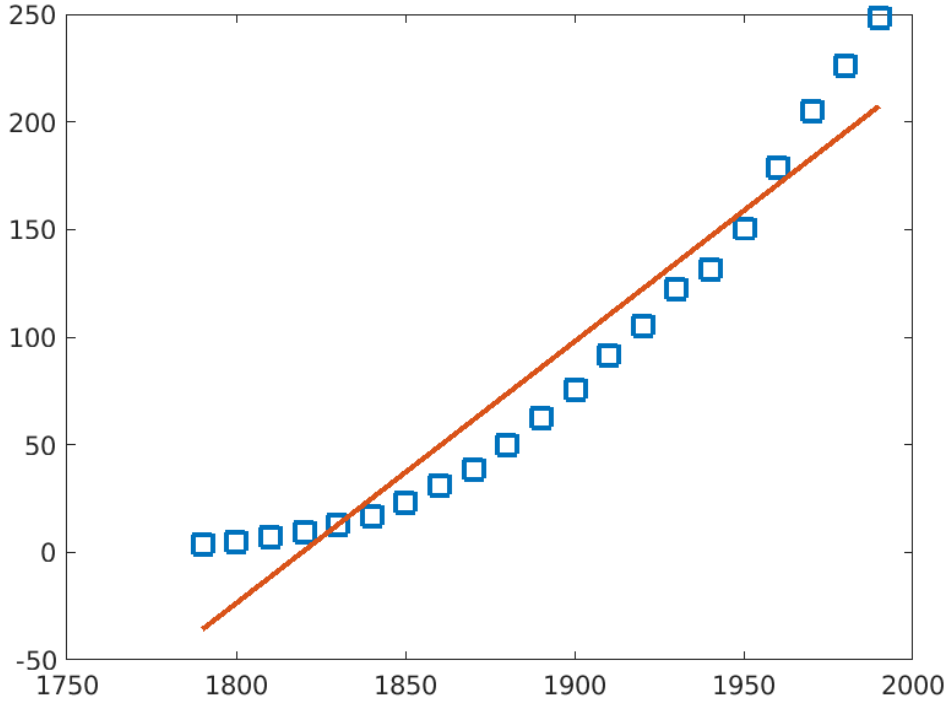
USCensusLeastSquares.m

```
load census;
p1 = polyfit(cdate, pop, 1); %% NOTE: degree=1
t = linspace(1790, 1990,1001);
plot(cdate, pop, 's', t, polyval(p1,t),...
   'LineWidth', 3, 'MarkerSize',10);
Diff1 = norm(pop - polyval(p1,cdate));
dp1 = polyder(p1);
fprintf('p=1, Error=%5.2f, Growth (1990)=%5.2f\n', ...
   Diff1, polyval(dp1, 1990))

Extrap = polyval(p1, 2000:10:2020);
Actual = [281.4, 308.7, 331.5];

fprintf('2000: pop estimate (actual) %.1f (%.1f)\n', ...
   Extrap(1), Actual(1));
fprintf('2010: pop estimate (actual) %.1f (%.1f)\n', ...
   Extrap(2), Actual(2));
fprintf('2020: pop estimate (actual) %.1f (%.1f)\n', ...
   Extrap(3), Actual(3));
```

2: Least Squares

Output:

```
p=1, Error=98.78, Growth Estimate (1990)= 1.22
2000: population estimate (actual) 219.5 (281.4)
2010: population estimate (actual) 231.6 (308.7)
2020: population estimate (actual) 243.8 (331.5)
```

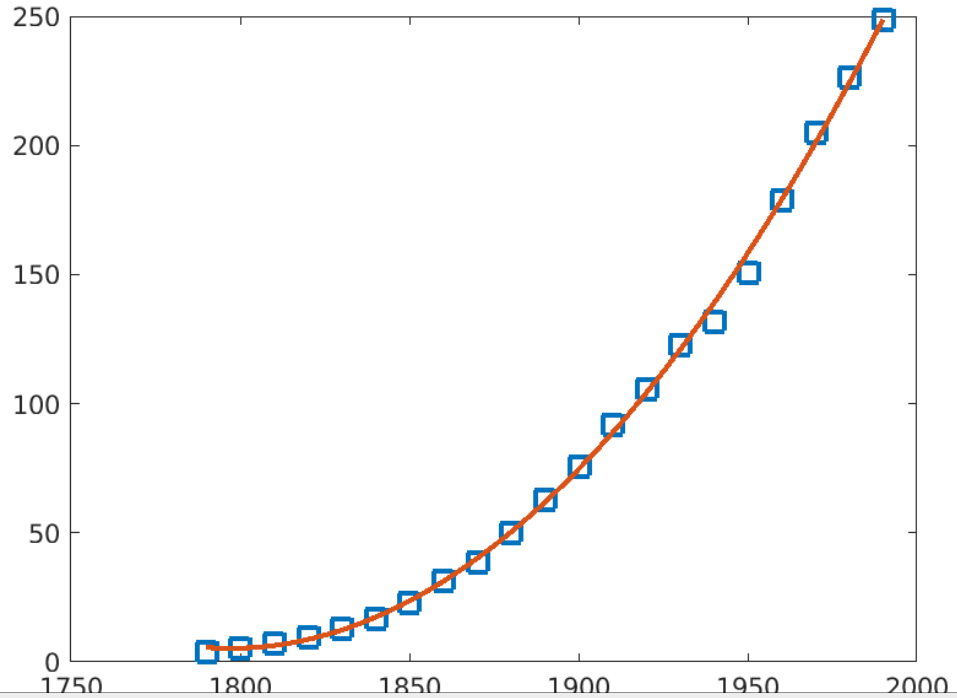


2: Least Squares

You can change the `polyfit` line to get a higher-order estimate.

The result for a quadratic would be: Output:

```
p=2, Error=12.61, Growth Estimate (1990)= 2.52
2000: population estimate (actual) 274.6 (281.4)
2010: population estimate (actual) 301.8 (308.7)
2020: population estimate (actual) 330.3 (331.5)
```



2: Least Squares

Experiment with higher-order interpolation. Convince yourself that the quadratic fit is most appropriate.

Suppose we want to find the polynomial $p_2 = a_2x^2 + a_1x + a_0$, that fits some data. If it is to fit the point (x_i, y_i), that means

$$a_2x_i^2 + a_1x_i + a_0 = y_i.$$

With 3 unknowns we need 3 equations. So if we have three points is there is an exact solution. The equations would be

$$\begin{aligned} a_2x_1^2 + a_1x_1 + a_0 &= y_1 \\ a_2x_2^2 + a_1x_2 + a_0 &= y_2 \\ a_2x_2^2 + a_1x_3 + a_0 &= y_3. \end{aligned}$$

We could write this as a matrix-vector equation: $Xa = y$, i.e.,

$$\begin{bmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ x_3^2 & x_3 & 1 \end{bmatrix} \begin{bmatrix} a_2 \\ a_1 \\ a_0 \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ y_2 \end{bmatrix}$$

But in these problems we have many more equations than unknowns:

$$\begin{bmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ x_3^2 & x_3 & 1 \\ \vdots & \vdots & \vdots \\ x_n^2 & x_n & 1 \end{bmatrix} \begin{bmatrix} a_2 \\ a_1 \\ a_0 \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

Now there is no (exact) solution to this problem: there is no vector a for which $Xa = y$.

But we can find a solution that is "better" than the rest. For that we need some way to understand "norms". (See next slide).

[Stuff about norms, hand-written in class].

Recall again the problem is

$$\begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \\ \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

where $n > 3$. There is no solution to $Xa = y$. That is, $\|Xa - y\| > 0$ for all a.

The "least squares" solution is the one for which, the residual, $Xa - y$, is as small as possible.

In a linear algebra course, we would prove such a vector exists, and explain how to find it. But here we will just compute it:

```
X = [x.^0, x, x.^2];
a = X\y
```

Try this for the US Census data.

You should take `x=cdate`, `y=pop`. Compare the resulting a with the value of p computed by `polyfit()`.

Try this for the US Census data.

You should take `x=cdate`, `y=pop`. Compare the resulting a with the value of p computed by `polyfit()`.

In-class test

There will be an in-class test next week, 4pm Wed 23rd Feb.

- It is a test on MATLAB (not scientific computing)
- Open book: use anything on Blackboard or the MATLAB help.
- There will be questions on
  - Defining and using anonymous functions;
  - Writing scripts in function files.
  - Flow-of-control (`if`, `for`, `while`...)
- Input and (especially) output.
- Working with vectors and matrices.
- Graphics.

Include your answers in a single script file, except, if needed, for any file function. Each question would be in a separate "section" of the script, meaning that there is a line between the relevant sections of code that start with a double percent. (See solution to sample questions, or ask during the test).

Q1. Define an anonymous function to represent $f(x) = \sin(x^2)$. Create a plot of it that resembles, as close as possible, the following , the image in 1.

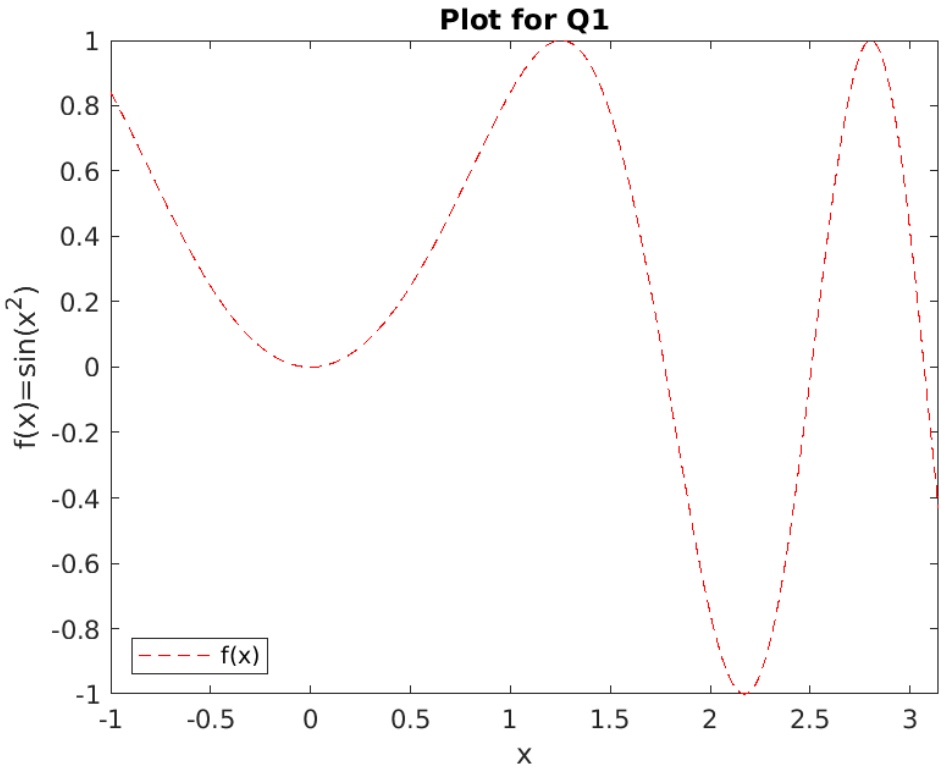


Figure: Figure for Q1

.

Q2. Write a (file) function called *make_normal()* that takes a single vector, x, as its input, and returns a vector y that is a scalar multiple of x but has the property that all its entries sum to 1. For example, if

$$x = \begin{bmatrix} 1 \\ 5 \\ 0 \\ -2 \\ 6 \end{bmatrix}, \quad \text{then it should return} \quad y = \begin{bmatrix} 0.1 \\ 0.5 \\ 0 \\ -0.2 \\ 0.6 \end{bmatrix}.$$

Your solution script file should demonstrate that your function works by applying it to the vector x above.

Q3. Find out what the `diff()` function does when applied to a vector. Write a script file that creates a **row** vector, x, such that `diff(x)` returns the vector $[0.1, 0.02, -3, 40, -500]$.

Q4. Write script that prompts the user to enter an integer m. Then set n to be the maximum of 5 and $\lfloor m \rfloor$. Next `for`-loops are used to make an $n \times n$ matrix with the property that

$$a_{i,j} = \begin{cases} i - j & n \text{ is even} \\ i + 2j & n \text{ is odd} \end{cases}$$

Q5. Show how to use the zeros function and the colon operator to make the matrix using only 3 lines of MATLAB (and no `for` loops or `if` statements).

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 2 & 2 & 2 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Q6. Consider the following lines of MATLAB code:

```
x = 0.1:0.01:1;
y = exp( -1./x);
for i=1:10:length(x)
    fprintf('i=%d, x=%f, y=%f\n', i, x(i), y(i));
end
```

Modify the `fprintf` line so that the output resembles, as closely as possible, the following:

```
i=  1, x= 0.1, y= 4.540e-05
i= 11, x= 0.2, y= 6.738e-03
i= 21, x= 0.3, y= 3.567e-02
i= 31, x= 0.4, y= 8.208e-02
i= 41, x= 0.5, y= 1.353e-01
i= 51, x= 0.6, y= 1.889e-01
i= 61, x= 0.7, y= 2.397e-01
i= 71, x= 0.8, y= 2.865e-01
i= 81, x= 0.9, y= 3.292e-01
i= 91, x= 1.0, y= 3.679e-01
```